IMAGE AND VIDEO PROCESSING

Mini-Project Report

Virtual Quiz Game using Real-Time Hand Tracking

A touch-free, webcam-based quiz controlled by the index fingertip (OpenCV + MediaPipe Hand Landmarker)

Project Title	Virtual Quiz Game using Real-Time Hand Tracking
Course	Image and Video Processing
Submitted BY	Rhytham Madnani Sparsh Bagdi
Roll Number	F029 F037
Programme / Semester / Division	Btech CE SEC-A SEM V
Submitted To	Dr. Raj Gaurav Mishra
Institute	NMIMS
Academic Year	2026 – 2027

Abstract

Conventional on-screen quizzes need a mouse, keyboard or touchscreen, which is unsuitable for hygiene-sensitive, classroom-projector or accessibility-oriented settings. This project builds a **touch-free multiple-choice quiz** that a player controls by pointing their index finger at a webcam. Each frame is captured with OpenCV, mirrored, converted from BGR to RGB and passed to the **MediaPipe Hand Landmarker**, which returns 21 hand landmarks; landmark 8 (index fingertip) is converted to pixel coordinates and stabilised with **exponential moving-average (EMA) smoothing** ($\alpha = 0.25$). A geometric hit-test against four answer regions and a 1.5 s **dwell-time** trigger confirm a selection, while a 20 s per-question timer, a live score, audio cues and an on-frame graphical overlay complete the game. Questions are loaded from a CSV file. The prototype, written in Python, runs the whole pipeline in a single real-time loop at camera frame rate and delivers a complete play-through (question, selection, feedback, final score, restart) using only a hand gesture. The report documents the design, parameters, timing model and limitations.

1. Introduction

Background. Human–computer interaction has progressively moved from keyboards and mice towards natural interfaces such as touch, voice and gesture. Vision-based gesture recognition lets a device understand a user with nothing more than an ordinary camera. At the same time, learning games and quizzes are widely used in classrooms and events to improve engagement and recall.

Relevance. A gesture-driven quiz removes the need for shared physical input devices (useful where hygiene matters), works with a projector and a webcam at a distance, can be used by people who find a mouse or keyboard difficult, and is far more engaging than a static form. Hand tracking is also the basis of many AR/VR, kiosk and smart-display interfaces, so the techniques transfer directly to other applications.

Role of image/video processing. The whole system depends on processing a live video stream: frames are acquired and pre-processed (flipping, colour-space conversion), the hand is detected and its landmarks are located by a machine-learning based detector, the temporal sequence of landmark positions is filtered, and results are rendered back onto the frames using drawing primitives. Video-mode tracking exploits temporal continuity between frames so the hand need not be re-detected from scratch every frame.

Motivation. The group wanted a project that combines the classical operations learnt in the course (colour conversion, geometric transformation, drawing/overlay, temporal filtering) with a modern landmark-detection model, and that gives a visible, interactive result which can be demonstrated live rather than only as numbers on a graph.

2. Problem Statement and Objectives

Real-world problem. Enable a user to answer multiple-choice questions without touching any device, using only a hand gesture captured by a standard webcam, in real time and with reliable, accidental-selection-resistant behaviour.

The objective of this project is to design and implement an interactive quiz game that tracks the user's index fingertip in a live video stream using image-processing and landmark-detection techniques and converts fingertip dwell on an answer region into a selection.

Specific objectives:

- Acquire and pre-process a live webcam stream (mirroring, BGR→RGB conversion) for real-time processing.
- Detect one hand and extract the index-fingertip landmark per frame using the MediaPipe Hand Landmarker in VIDEO mode.
- Stabilise the fingertip trajectory with temporal (EMA) smoothing and map it to on-screen answer regions via a geometric hit-test.
- Implement a dwell-based selection (1.5 s) with a visible progress bar, a 20 s per-question timer, score keeping and audio/visual feedback.
- Load questions from an editable CSV file and support restart / quit so that the game is reusable with new content.

3. Scope of the Project

What the project covers

- Real-time single-hand fingertip tracking and pointer control from a laptop/USB webcam.
- Full quiz flow: question display, four-option answering, timeout handling, score, final screen and restart.
- Temporal smoothing, dwell-time selection, and an overlay user interface drawn with OpenCV primitives.
- Content-driven questions through `questions.csv` (currently five general-knowledge questions).

What it does not cover

- Gesture vocabularies beyond pointing (pinch, swipe, fist), two-hand interaction, or multi-player play.
- Training or fine-tuning a hand-detection model – the pre-trained MediaPipe model is used as is.
- Question authoring tools, a database, user accounts, or network/online play.

- Formal usability studies with large groups of users.

Type of video used: live RGB webcam video of a person's upper body and hand, captured indoors. **Application domain:** education and gamification, touch-free kiosks/public displays, accessibility, and gesture-based HCI demonstrations.

4. Dataset / Input Description

The system is a live-input application and does **not use a pre-recorded image or video dataset**; the hand-detection model was already trained by its authors (Google). The inputs are therefore a live camera stream and a question file, summarised below.

Item	Description
Video source	Default webcam, device index 0, opened with `cv2.VideoCapture(0, cv2.CAP_DSHOW)` (DirectShow backend, Windows)
Number of videos / images	One continuous live stream; no stored images. Each frame is processed independently by the landmarker with temporal tracking between frames
Resolution	Camera-native (not forced in code); the frame size is read at run time and all UI coordinates are computed as fractions of width/height. Typical webcam modes are 640×480 or 1280×720
Frame format / colour	8-bit, 3-channel BGR from OpenCV; converted to 8-bit SRGB for MediaPipe
Frame rate / duration	Camera dependent (typically ~30 FPS). Session duration: 15 s (fastest possible play-through) to 107.5 s (all five questions timed out)
Pre-trained model	`hand_landmarker.task` – MediaPipe Hand Landmarker bundle (7.82 MB): palm detector + 21-point landmark regressor
Question file	`questions.csv`, UTF-8 with BOM, 5 rows, columns: question, option1–option4, answer (1–4)
Audio asset	`yay.mp3` (49 KB) – played on completion via pygame; beeps via winsound

Table 1. *Question bank (`questions.csv`).*

#	Question	Options (1–4)	Correct
1	What is the capital of India?	Delhi, Mumbai, Pune, Jaipur	1 (Delhi)
2	Which planet is known as the Red Planet?	Earth, Mars, Jupiter, Venus	2 (Mars)
3	What is 5 + 5?	8, 9, 10, 11	3 (10)
4	Which language is used for Android development?	HTML, Python, Java, SQL	3 (Java)
5	What is the largest ocean?	Indian, Atlantic, Pacific, Arctic	3 (Pacific)

Sample input frame. Because the input is a live webcam stream containing the player, no stored sample frames are bundled with the project. The interface frames in Fig. 5 (Section 8) show how a captured frame is annotated; they were rendered with the project's own drawing routine over a synthetic backdrop that stands in for the camera image.

5. Methodology / Proposed Approach

The solution is a single real-time loop (Fig. 1 and Fig. 2). For every frame the program: (1) reads and mirrors the frame so that on-screen motion matches the user's hand; (2) converts it to RGB and runs the MediaPipe Hand Landmarker with a monotonically increasing timestamp (+33 ms per frame ≈ 30 FPS); (3) takes landmark 8, the index fingertip, scales its normalised (x, y) to pixels and applies EMA smoothing; (4) tests whether the smoothed point lies in one of four rectangles; (5)

runs the dwell timer, and when it reaches 1.5 s checks the choice against the answer key; (6) draws the interface and displays it. Timeout, feedback delay and question progression run on a monotonic clock.

Fig. 1 Overall processing pipeline

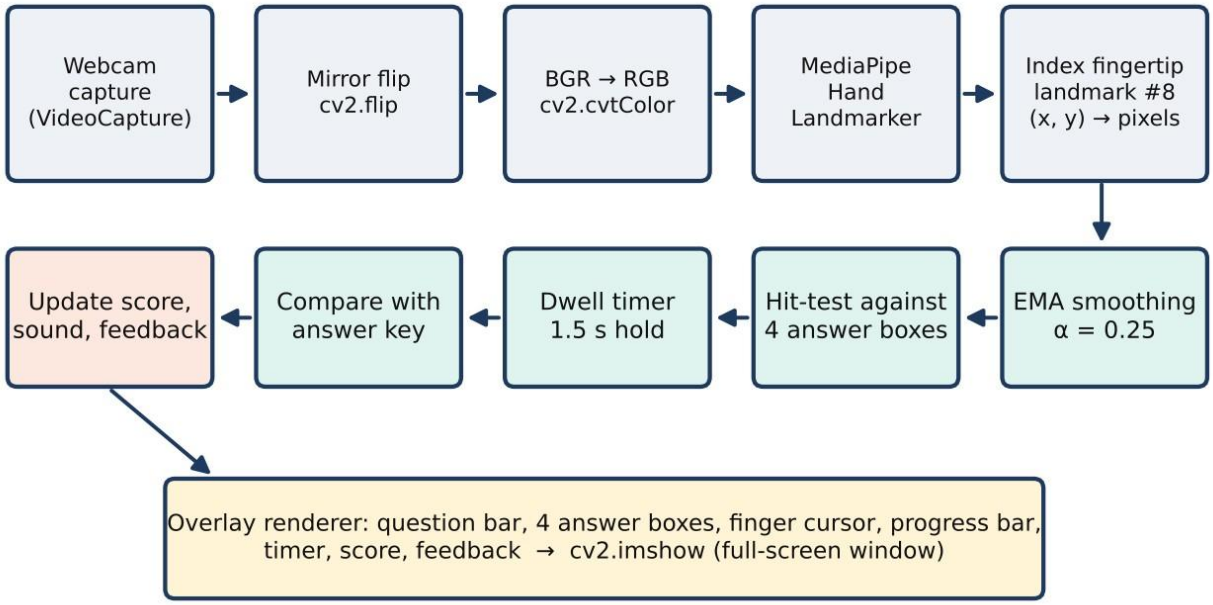


Fig. 1 summarises the data flow from camera to feedback.

Fig. 2 Game-loop flowchart (executed once per camera frame)

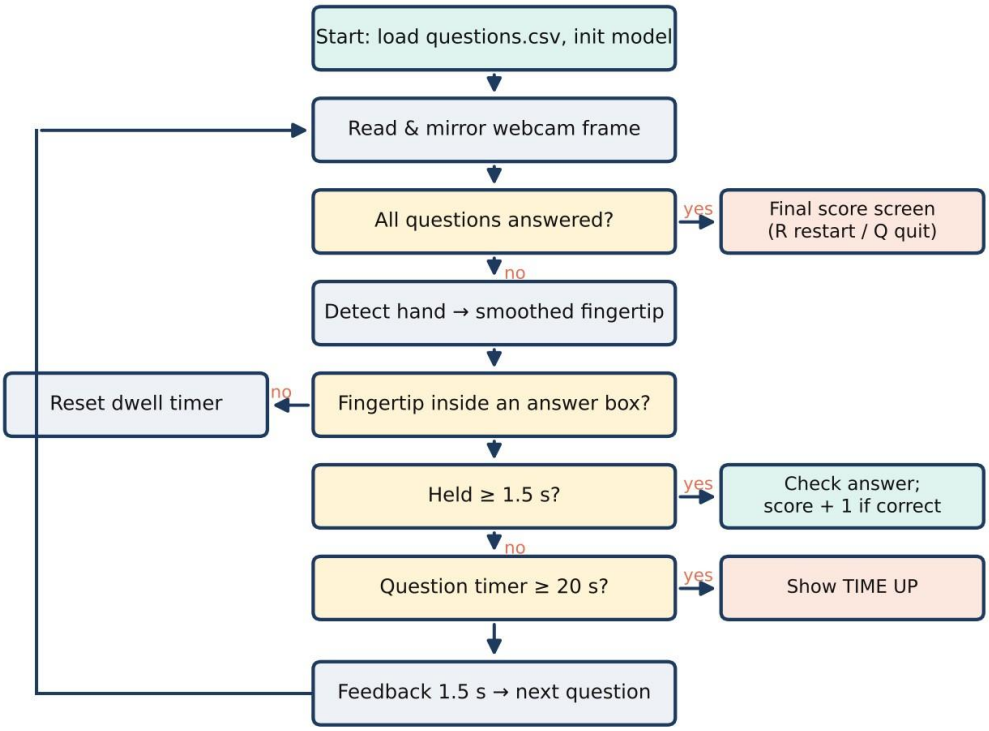


Fig. 2 gives the per-frame decision logic.

Algorithm (pseudocode)

```
load questions; create HandLandmarker(VIDEO, num_hands=1, conf=0.5)
q = 0; score = 0; t_q = now() while camera.read(frame):
```

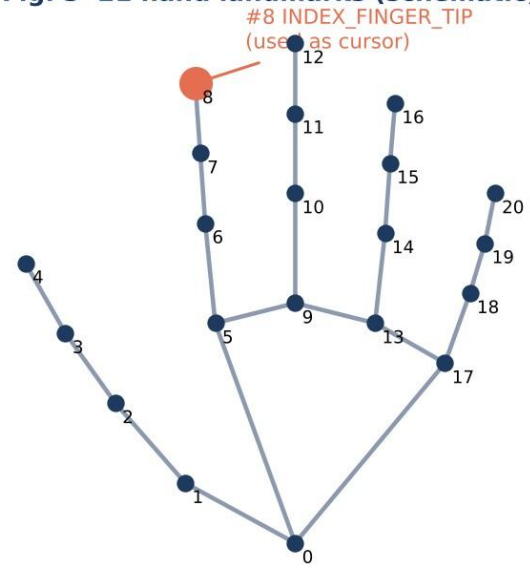
```
    frame = flip(frame); rgb = BGR2RGB(frame)      if q
== N: show final score; handle R/Q; continue      hand
= landmarker.detect_for_video(rgb, ts += 33)
remaining = 20 - (now() - t_q)
    draw header(question, score, remaining) and 4 option boxes
if hand:
        (x, y) = pixels(hand[8]);  s = s + 0.25 * ((x, y) - s)     # EMA
h = box containing s (or None)          if h != hovered: hovered = h;
t_hover = now()          if h and not feedback:
            draw progress(now() - t_hover / 1.5)             if now() - t_hover
>= 1.5: score += (h+1 == answer); feedback = ...
    if feedback and now() - t_fb > 1.5: q += 1; reset timers
if remaining == 0 and not feedback: feedback = 'TIME UP'
imshow(frame)
```

6. Techniques / Algorithms Used

Image processing

Technique	Where / how it is used
Colour-space conversion	`cv2.cvtColor(BGR→RGB)` to match the channel order expected by MediaPipe
Geometric transform (mirroring)	`cv2.flip(frame, 1)` gives a natural mirror view so cursor motion agrees with hand motion
Machine-learning hand detection and feature extraction	MediaPipe Hand Landmarker: a palm detector locates the hand, then a landmark model regresses 21 3-D keypoints (Fig. 3). Landmark 8 (INDEX_FINGER_TIP) is the feature used as pointer
Coordinate mapping	Normalised landmark (x, y) ∈ [0,1] multiplied by frame width/height gives pixel position
Technique	**Where / how it is used**
Temporal low-pass filtering	EMA: $s = s + \alpha(x - s)$, $\alpha = 0.25$, applied to x and y to suppress t_{t-1} t_{t-1} landmark jitter
Region-of-interest hit-testing	Axis-aligned rectangle containment test of the smoothed fingertip against four answer boxes
Drawing / overlay	`rectangle, putText, circle` render header, boxes, highlight, progress bar, cursor and feedback text on the frame

Fig. 3 21 hand landmarks (schematic)



The 21 landmark indices produced by the hand model; index 8 drives the game.

Video processing

- **Frame extraction:** every frame of the live stream is acquired with `cap.read()` and processed individually.
- **Tracking in video mode:** RunningMode.VIDEO with min detection/presence/tracking confidence of 0.5 lets the model track the hand between frames using increasing timestamps, avoiding full re-detection every frame.
- **Temporal filtering and dwell logic:** smoothing across frames and a time-based hover trigger convert noisy per-frame positions into a stable, intentional selection.

Background subtraction, optical flow and frame differencing were not required because the hand is localised by the learned landmark model rather than by motion cues.

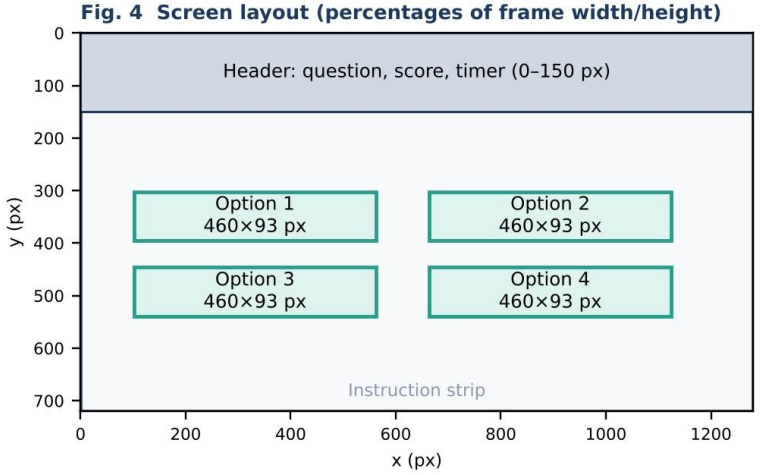
7. Implementation

Aspect	Details
Language	Python 3
Libraries / frameworks	OpenCV (capture, colour conversion, drawing, display), MediaPipe Tasks API (HandLandmarker), pygame (mp3 playback), winsound (beeps), csv, time, os (standard library)
Software / tools	Windows, any Python IDE, webcam driver (DirectShow); model file hand_landmarker.task
Hardware	Laptop/PC with built-in or USB webcam; CPU inference (no GPU code); speakers for audio cues
Source files	`main.py` (797 lines, the game), `camera_test.py` (25 lines, webcam check), `questions.csv`, `hand_landmarker.task`, `yay.mp3`

Important implementation steps

- **Camera check:** `camera_test.py` opens the webcam, shows the mirrored feed and exits on Q – used to verify the device before running the game.
- **Setup:** read the CSV (`utf-8-sig`), build `HandLandmarkerOptions` (VIDEO mode, one hand, thresholds 0.5), open the camera and create a full-screen window.
- **Layout:** answer boxes use proportions of the frame (margin 8 %, box width 36 %, height 13 %, gap 8 %, rows at 42 % and 62 % of height) so the UI scales to any resolution (Fig. 4).
- **Selection logic:** hover start time is stored when the fingertip enters a box and reset when it leaves; a yellow bar grows across the box width and the answer is committed at 1.5 s.
- **Game state:** score, question index, 20 s countdown (red and a beep for the last 5 s), 1.5 s feedback pause, TIME UP handling, final screen with R (restart) and Q (quit).

Fig. 4 Screen layout (percentages of frame width/height)



Layout of the UI computed at a 1280×720 frame.

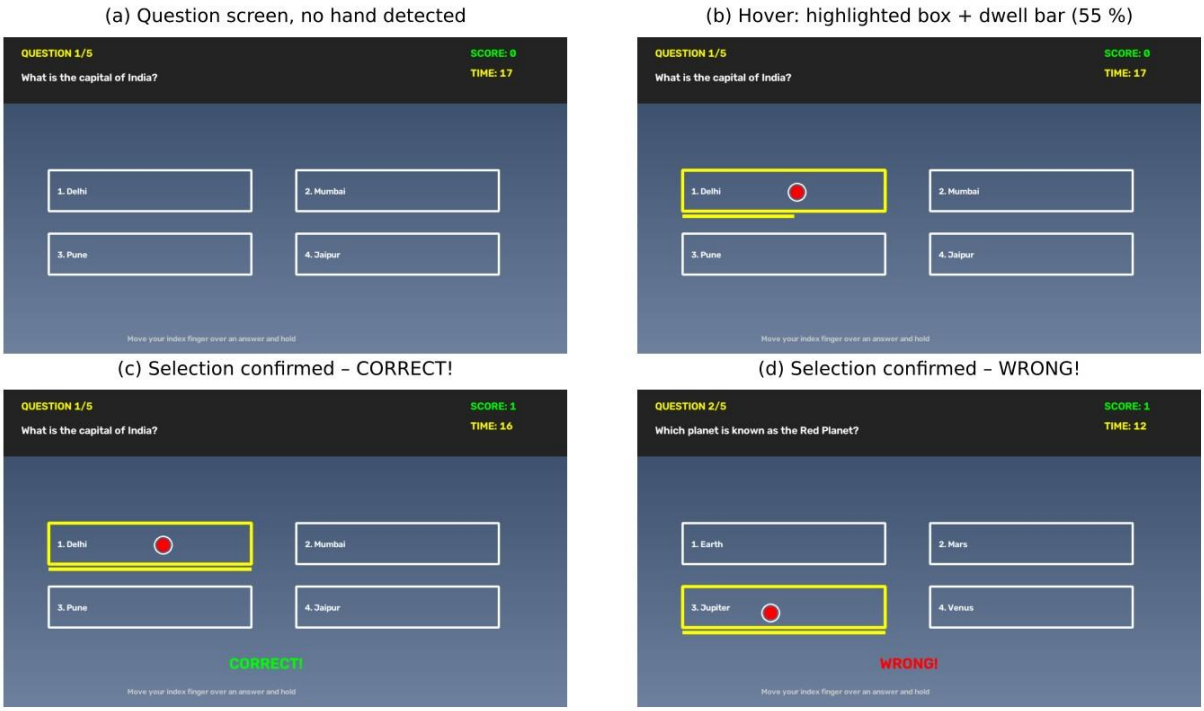
Key parameters

Parameter	Value	Purpose
QUESTION_TIME	20 s	Time allowed per question
SELECTION_TIME	1.5 s	Dwell needed to confirm an answer
SMOOTHING (α)	0.25	EMA weight of the newest sample
num_hands	1	Track a single player's hand
min detection / presence / tracking confidence	0.5 each	Landmarker thresholds
Feedback delay	1.5 s	Pause showing CORRECT / WRONG / TIME UP
Timestamp step	33 ms	Synthetic video timestamps (≈30 FPS)

8. Results and Discussion

The prototype completes a full quiz session driven only by the fingertip. This section presents the interface outputs, the behaviour implied by the chosen parameters, and quantitative measures derived from the implementation. Interface figures were rendered with the project's drawing code at 1280×720 on a synthetic backdrop, and the smoothing figure is a simulation using the project's α; both are labelled as such.

Fig. 5 Game interface rendered with the project's drawing routine (1280×720, synthetic backdrop)

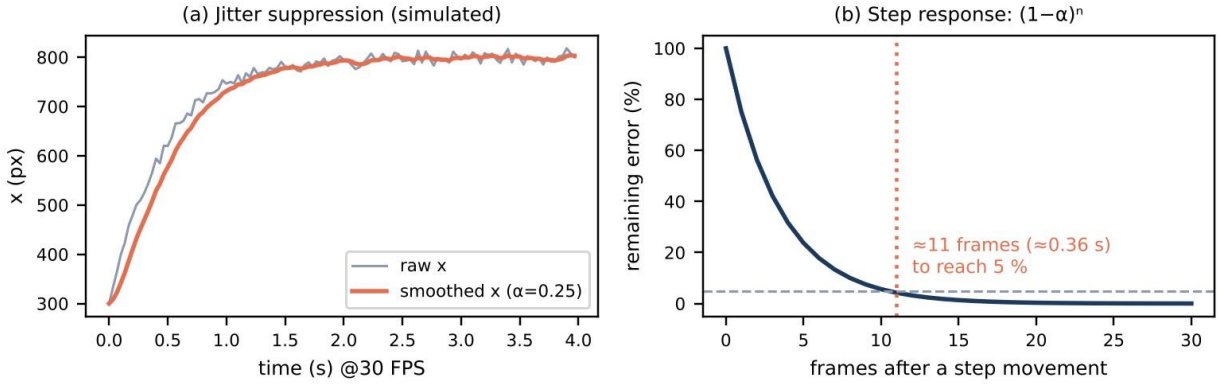


Input → output: a frame with no hand (a), the hover state with highlighted box and dwell progress bar (b), and the confirmed outcomes (c, d).

Intermediate processing results

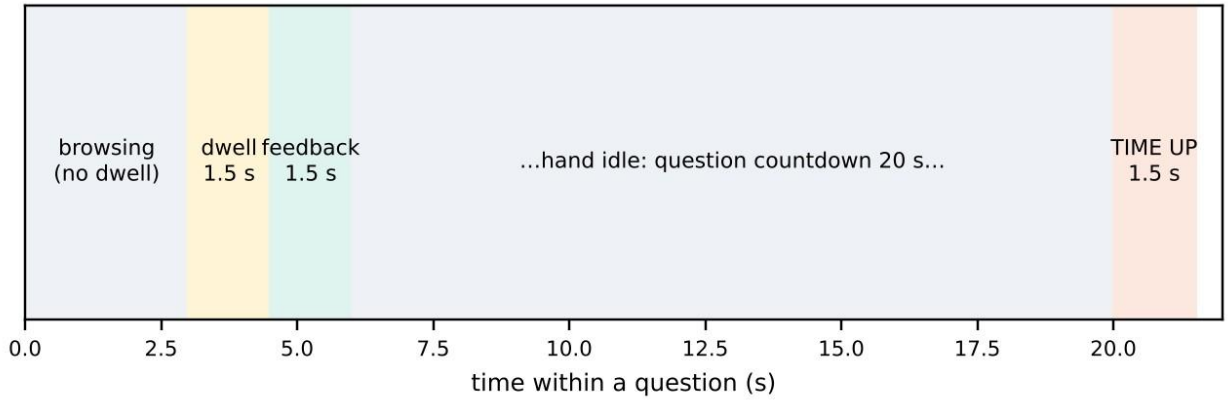
The intermediate quantities are the 21 landmarks (Fig. 3), the raw fingertip pixel position, the smoothed position (red cursor with white ring, Fig. 5b) and the hover state (yellow border and bar). Smoothing is the key intermediate step: Fig. 6 shows that with α = 0.25, a simulated fingertip path with ±9 px (σ) sensor noise has its frame-to-frame step standard deviation reduced from 13.7 px to 6.9 px, roughly halved, which makes the cursor steadier and the 1.5 s dwell easier to hold inside a box.

Fig. 6 Effect of exponential smoothing on the fingertip cursor



Effect of EMA smoothing (simulation with the project's α and 30 FPS). Smoothing introduces a small lag: 95 % of a step is reached in about 11 frames (≈0.36 s).

Fig. 7 Timing model of one question (answered vs. timed-out)



Timing model of one question derived from the constants in the code.

Quantitative design measures

Measure	Value	How obtained
Dwell frames needed	≈45 frames at 30 FPS (1.5 s)	SELECTION_TIME × FPS
Cursor settling time (95 %)	≈11 frames ≈ 0.36 s	$(1-\alpha)^n \le 0.05 \rightarrow n \ge \ln 0.05 / \ln 0.75$
Per-question time budget	20 s answering + 1.5 s feedback	Constants in code
Fastest / slowest full session	15 s / 107.5 s (5 questions)	5×(1.5+1.5) / 5×(20+1.5)
Possible scores	0 – 5	One point per correct answer
Answer box size @ 1280×720	460 × 93 px (36 % × 13 %)	Layout formula
Answer box size @ 640×480	230 × 62 px	Layout formula
Gap between boxes @ 1280×720	102 px horizontal, 51 px vertical	Layout formula
Jitter (step σ), simulated	13.7 px → 6.9 px	Fig. 6 simulation

Discussion. The landmark-based approach needs no markers, gloves or colour thresholds, and the dwell trigger avoids accidental selection when the finger merely passes over an option. The 1.5 s dwell and 0.25 smoothing were chosen as a trade-off: shorter dwell or weaker smoothing responds faster but triggers accidentally; longer dwell or stronger smoothing is steadier but feels slower. Using proportional layout keeps the boxes large enough (≥62 px high even at 480p) for comfortable pointing at arm's length. Reproducing measured live-camera accuracy, latency and frame-rate figures requires running `main.py` on the target machine; the values above are design-level quantities derived from the code.

9. Challenges / Limitations

- **Platform dependence:** `winsound` and `CAP_DSHOW` make the program Windows-only; Linux/macOS require replacing both.
- **Blocking audio:** `winsound.Beep` blocks the loop for 0.1–0.55 s, so the video momentarily freezes at feedback and countdown beeps.
- **Landmark jitter and lag:** raw fingertip positions are noisy; smoothing removes jitter but adds ≈0.36 s of settling lag during fast movement.
- **Difficult cases:** poor or back-lighting, very fast motion, a hand partly out of frame, a cluttered or skin-coloured background, or the hand too far/near the camera can lose detection; the cursor then disappears and no selection is possible.
- **Selection ambiguity:** a resting finger over a box will select it after 1.5 s; there is no 'undo' and no explicit confirm gesture.

- **Dataset/content limits:** only five questions, text-only, with long questions wrapped to two lines only (about 130 characters).
- **Computational limits:** inference runs on CPU in the same thread as drawing, so older machines may drop below 30 FPS, while timestamps are advanced by a fixed 33 ms regardless of the actual rate.
- **Evaluation:** accuracy and latency were not measured with a labelled test set or a user study.

10. Conclusion

Implemented: a complete, CSV-driven, real-time multiple-choice quiz that is operated by pointing a finger at a webcam. **Techniques used:** frame acquisition, mirroring and BGR→RGB conversion; MediaPipe Hand Landmarker (palm detection plus 21-point landmark regression) in video mode; coordinate mapping; EMA temporal smoothing; rectangle hit-testing; dwell-time selection; and OpenCV overlay rendering with audio feedback.

Achieved: a touch-free interaction pipeline that runs in a single loop, includes timers, scoring, restart and quit, and has a layout that adapts to any camera resolution. **Learning:** the group learnt how a deployable vision pipeline is assembled from acquisition, pre-processing, model inference, temporal filtering and rendering; how filtering and dwell time trade responsiveness against stability; and how practical issues such as lighting, platform-specific APIs and blocking calls affect real-time systems. Future work: replace blocking sounds with non-blocking audio, add a pinch-to-select gesture, make the code cross-platform, enlarge the question bank, and measure accuracy and latency in a user study.

References

[1] C. Lugaresi et al., “MediaPipe: A Framework for Building Perception Pipelines,” arXiv:1906.08172, 2019.

[2] F. Zhang et al., “MediaPipe Hands: On-device Real-time Hand Tracking,” arXiv:2006.10214, 2020.

[3] Google AI Edge, “Hand landmarks detection guide (MediaPipe Tasks, HandLandmarker),” ai.google.dev/edge/mediapipe.

[4] G. Bradski, “The OpenCV Library,” Dr. Dobb's Journal of Software Tools, 2000; docs.opencv.org.

[5] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[6] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[7] Pygame documentation, pygame.org/docs (pygame.mixer.music).